Logische poorten

python.exposed
Learn Python to Automate, Analyze, Accelerate.

Logische poorten

AND Gate



Waarheidstabel

Ingang A	Ingang B	Uitgang (A AND B)
0	0	0
0	1	0
1	0	0
1	1	1

Logische poorten

```python
def AND_gate(a, b):
    return a & b

# Testen
print(AND_gate(0, 0))  # Output: 0
print(AND_gate(0, 1))  # Output: 0
print(AND_gate(1, 0))  # Output: 0
print(AND_gate(1, 1))  # Output: 1
```


Python.exposed Learn Python to Automate, Analyze, Accelerate.

Logische poorten

OR Gate



Waarheidstabel

Ingang A	Ingang B	Uitgang (A OR B)
0	0	0
0	1	1
1	0	1
1	1	1

Logische poorten

```python
def OR_gate(a, b):
    return a | b

# Testen
print(OR_gate(0, 0))  # Output: 0
print(OR_gate(0, 1))  # Output: 1
print(OR_gate(1, 0))  # Output: 1
print(OR_gate(1, 1))  # Output: 1
```


python.exposed
Learn Python to Automate, Analyze, Accelerate.

Logische poorten

NOT Gate



Waarheidstabel

Ingang A	Uitgang (NOT A)
0	1
1	0

Logische poorten

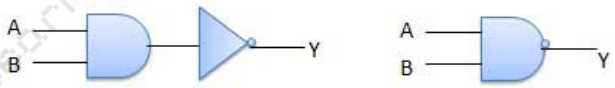
```python
def NOT_gate(a):
    return not a

# Testen
print(NOT_gate(0))  # Output: True (1)
print(NOT_gate(1))  # Output: False (0)
```


python.exposed
Learn Python to Automate, Analyze, Accelerate.

Logische poorten

NAND (NOT-AND) Gate



Waarheidstabel

Ingang A	Ingang B	Uitgang (A NAND B)
0	0	1
0	1	1
1	0	1
1	1	0

Logische poorten

```python
def NAND_gate(a, b):
    return not (a and b)

# Testen
print(NAND_gate(0, 0))  # Output: True (1)
print(NAND_gate(0, 1))  # Output: True (1)
print(NAND_gate(1, 0))  # Output: True (1)
print(NAND_gate(1, 1))  # Output: False (0)
```


python.exposed learn python to automate, analyze, accelerate.

Logische poorten

NOR (NOT-OR) Gate



Waarheidstabel

Ingang A	Ingang B	Uitgang (A NOR B)
0	0	1
0	1	0
1	0	0
1	1	0

Logische poorten

```python
def NOR_gate(a, b):
    return not (a or b)

# Testen
print(NOR_gate(0, 0))  # Output: True (1)
print(NOR_gate(0, 1))  # Output: False (0)
print(NOR_gate(1, 0))  # Output: False (0)
print(NOR_gate(1, 1))  # Output: False (0)
```


python.exposed
learn python to automate, analyze, accelerate.

Logische poorten

XOR (Ex-OR) Gate



Waarheidstabel

Ingang A	Ingang B	Uitgang (A XOR B)
0	0	0
0	1	1
1	0	1
1	1	0

Logische poorten

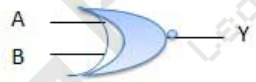
```python
def XOR_gate(a, b):
    return a != b

# Testen
print(XOR_gate(0, 0))  # Output: False (0)
print(XOR_gate(0, 1))  # Output: True (1)
print(XOR_gate(1, 0))  # Output: True (1)
print(XOR_gate(1, 1))  # Output: False (0)
```


python.exposed
Learn Python to Automate, Analyze, Accelerate.

Logische poorten

XNOR Gate



Waarheidstabel

Ingang A	Ingang B	Uitgang (A XNOR B)
0	0	1
0	1	0
1	0	0
1	1	1

Logische poorten

```python
def XNOR_gate(a, b):
    return a == b

# Testen
print(XNOR_gate(0, 0))  # Output: True (1)
print(XNOR_gate(0, 1))  # Output: False (0)
print(XNOR_gate(1, 0))  # Output: False (0)
print(XNOR_gate(1, 1))  # Output: True (1)
```


python.exposed
Learn Python to automate, analyze, accelerate.

Algoritmes

Wat is een algoritme?

Een algoritme is een reeks logische stappen om een taak uit te voeren. In de informatica gebruiken we algoritmes om computerprogramma's te maken.

Instructie: Eén stap binnen een algoritme die een computer kan uitvoeren.

Voorbeelden van logische stappen:

- Naar een pretpark gaan: Welke stappen moet je nemen?
- Een product kopen in een webshop: Welke keuzes maak je?
- Inschrijven voor een cursus: Wat zijn de noodzakelijke acties?

Logica in programmeren

Programmeren is het opstellen van deze logische stappen (algoritmes) zodat een computer precies weet wat hij moet doen.

Voorbeeld van logische regels:

- Als aan een voorwaarde wordt voldaan, doe dit.
- Zo niet, doe dat.

python.exposed
learn python to automate, analyze, accelerate.

Logica in programmeren

- Algoritmes lossen problemen op door stappen en regels te volgen. Deze regels helpen bepalen welk pad te kiezen wanneer er een beslissing nodig is.
- Soms zijn er meerdere oplossingen voor een probleem. Voor elke oplossing kan een nieuw algoritme gemaakt worden.
- Logisch redeneren helpt voorspellen wat er gebeurt bij elke stap in het algoritme. Voorspellingen kunnen gebruikt worden om oplossingen te vergelijken en de beste te kiezen.

Algoritmes ontwerpen

- Een algoritme is een plan, een reeks logische stappen om een probleem op te lossen. Algoritmes worden vaak beschreven in stroomschema's of in pseudocode.
- Stuur je denkproces stap voor stap om het probleem te begrijpen en op te lossen.
  - 'Wat als we het anders aanpakken?' Door verschillende manieren te verkennen, vind je de beste oplossing.
- Bij het ontwerpen van een algoritme is het nuttig om te bedenken of er meerdere manieren zijn om het probleem aan te pakken.

Algoritmes ontwerpen

- Bij het ontwerpen van een algoritme moet je twee dingen in gedachten houden:
  - Het grote plaatje: Wat is het uiteindelijke doel?
  - Individuele stappen: Welke obstakels moet je oplossen om dat doel te bereiken?
- Voordat je een algoritme kunt ontwerpen, is het belangrijk het probleem volledig te begrijpen. Stel jezelf deze vragen:
  - Wat zijn de invoer- en uitvoerwaarden van het probleem?
  - Welke volgorde moeten de instructies volgen?
  - Welke beslissingen moeten genomen worden?
  - Worden bepaalde delen van het probleem herhaald?
- Als je deze basis begrijpt, kun je beginnen met het ontwerpen van het algoritme.

Pseudocode

- Pseudocode is een manier om een reeks instructies voor een computerprogramma te beschrijven in eenvoudige taal, zoals Engels of Nederlands. Het helpt om een programma te plannen voordat het geprogrammeerd wordt.
- Waarom pseudocode gebruiken?
  - Programma's worden geschreven in programmeertalen met een specifieke syntax.
  - Pseudocode is geen programmeertaal. Het is een eenvoudige manier om instructies te beschrijven zonder de exacte syntax van een programmeertaal te gebruiken.

python.exposed
Learn Python to Automate, Analyze, Accelerate.